

```

class Account{
    private String accountnumber; // details are hide
    private String accountholder;
    public double balance;
    public Account(String accountnumber , String accountholder ,double balance){
        this.accountnumber = accountnumber; //using constructors for parameters
        passing
        this.accountholder = accountholder;
        this.balance = balance;
    }
    public String getaccountnumber() // doing encapsulation by setters and getters
    {
        return accountnumber;
    }
    public String getaccountholder(){
        return accountholder;
    }
    public double getbalance(){
        return balance;
    }
    void deposit (double amount){
        if(amount <= 0){
            System.out.println("deposit amount is greater than zero");
            return;
        }
        balance += amount;
        System.out.println(" the deposited amount is " + amount);
    }
}

```

```

        void withdraw (double amount){
};

        void calinterest(){};

        void transfer(Account fixedAccount , double amount){

System.out.println("transfer
of"+amount+"from"+accountnumber+"to"+fixedAccount.getaccountnumber()+"---");

        this.withdraw(amount);

        fixedAccount.deposit(amount);

        }

        void show(){

System.out.println("ACCOUNT NUMBER IS " + accountnumber);

System.out.println("ACCOUNT HOLDER IS " + accountholder);

System.out.println("BALANCE IS " + balance);

        }

        }

class SavingsAccount extends Account {

        private double interestrate;

        public SavingsAccount(String accountnumber , String accountholder , double
balance){

                super(accountnumber, accountholder, balance);

                this.interestrate = interestrate;

        }

        @Override

        void withdraw(double amount){

                if(amount <= 0){

                        System.out.println("with draw amount is not possible ");

                }else if (amount >= balance){

                        System.out.println("it has insufficient balance");

                }else{

```

```

        balance -= amount;

        System.out.println("withdraw is " + amount );
    }
}

@Override
void calinterest(){
    double interest = balance *interestrate;

    balance += interest;

    System.out.println("interest is credited" + interest);
}
}

class CurrentAccount extends Account{
    private double overdraftlimit;

    public CurrentAccount(String accountnumber , String accountholder , double
balance){
        super(accountnumber, accountholder, balance);

        this.overdraftlimit = overdraftlimit;
    }

@Override
void withdraw(double amount){
    if(amount <= 0)

        System.out.println("with draw amount is not possible ");

    else if (amount > balance + overdraftlimit){
        System.out.println("it has insufficient balance and the limited is done");
    }else{
        balance -= amount;

        System.out.println("withdraw is " + amount );
    }
}

```

```

    }

@Override
void calinterest(){
    double charge = 50.0;

    balance += charge;

    System.out.println("maintainence charge is credited" + charge);
}
}

public class LabProgram1 {

    public static void main(String[] args) {

        CurrentAccount ca = new CurrentAccount("UB 24311", "kumar", 4500);
        SavingsAccount sa = new SavingsAccount("sb 24533", "heamnth", 5000);

        System.out.println(" initial account details");

        ca.show();

        System.out.println("");

        sa.show();


        System.out.println("transaction details");

        ca.deposit(2000);

        ca.withdraw(2500);

        ca.calinterest();

        sa.withdraw(3000);

        sa.calinterest();

        sa.transfer(ca, 1000);

        System.out.println("final account details");

        ca.show();

        System.out.println("");

        sa.show();
    }
}

```

}

}